

Feb22

Gao

2024-02-22

```
library(datasets)
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.1.1 --
```

```
## v broom      1.0.5    v recipes      1.0.9
## v dials      1.2.0    v rsample      1.2.0
## v dplyr      1.1.4    v tibble       3.2.1
## v ggplot2    3.4.4    v tidyr        1.3.1
## v infer      1.0.6    v tune         1.1.2
## v modeldata  1.3.0    v workflows    1.1.3
## v parsnip    1.1.1    v workflowsets 1.0.1
## v purrr      1.0.2    v yardstick    1.3.0
```

```
## -- Conflicts ----- tidymodels_conflicts() --
```

```
## x purrr::discard() masks scales::discard()
## x dplyr::filter()   masks stats::filter()
## x dplyr::lag()      masks stats::lag()
## x recipes::step()   masks stats::step()
## * Dig deeper into tidy modeling with R at https://www.tmr.org
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v forcats  1.0.0    v readr      2.1.5
## v lubridate 1.9.3    v stringr    1.5.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x readr::col_factor() masks scales::col_factor()
## x purrr::discard()    masks scales::discard()
## x dplyr::filter()     masks stats::filter()
## x stringr::fixed()    masks recipes::fixed()
## x dplyr::lag()        masks stats::lag()
## x readr::spec()       masks yardstick::spec()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rpart.plot)
```

```
## Loading required package: rpart
##
## Attaching package: 'rpart'
##
## The following object is masked from 'package:dials':
##
##     prune
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:yardstick':
##
##     precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(ggforce)
```

```
irises <- as_tibble(iris)
```

```
irises
```

```
## # A tibble: 150 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##   <dbl>         <dbl>         <dbl>         <dbl> <fct>
## 1         5.1         3.5         1.4         0.2 setosa
## 2         4.9         3         1.4         0.2 setosa
## 3         4.7         3.2         1.3         0.2 setosa
## 4         4.6         3.1         1.5         0.2 setosa
## 5         5         3.6         1.4         0.2 setosa
## 6         5.4         3.9         1.7         0.4 setosa
## 7         4.6         3.4         1.4         0.3 setosa
## 8         5         3.4         1.5         0.2 setosa
## 9         4.4         2.9         1.4         0.2 setosa
## 10        4.9         3.1         1.5         0.1 setosa
## # i 140 more rows
```

```
irises$Species %>% unique()
```

```
## [1] setosa    versicolor virginica
## Levels: setosa versicolor virginica
```

```
iris_split <- initial_split(irises, prop = 0.75)
```

```
train <- training(iris_split)
```

```
test <- testing(iris_split)
```

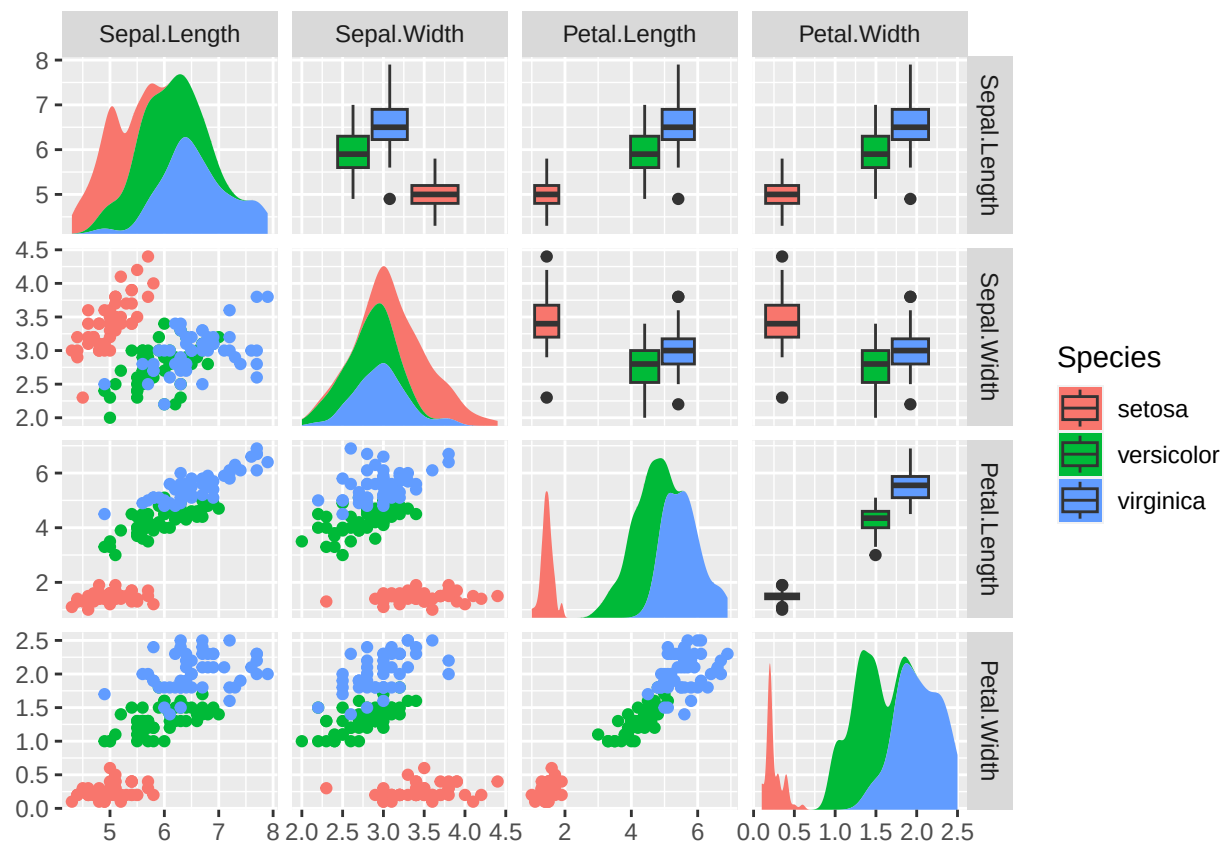
```
train
```

```
## # A tibble: 112 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##   <dbl>         <dbl>         <dbl>         <dbl> <fct>
## 1         5.7         3         4.2         1.2 versicolor
## 2         5.1         3.3         1.7         0.5 setosa
## 3         5         3.3         1.4         0.2 setosa
## 4         6.7         3         5.2         2.3 virginica
## 5         5.2         3.4         1.4         0.2 setosa
## 6         5.5         2.6         4.4         1.2 versicolor
## 7         5.4         3.4         1.5         0.4 setosa
## 8         5         3.4         1.5         0.2 setosa
## 9         5.4         3         4.5         1.5 versicolor
## 10        4.8         3.4         1.9         0.2 setosa
## # i 102 more rows
```

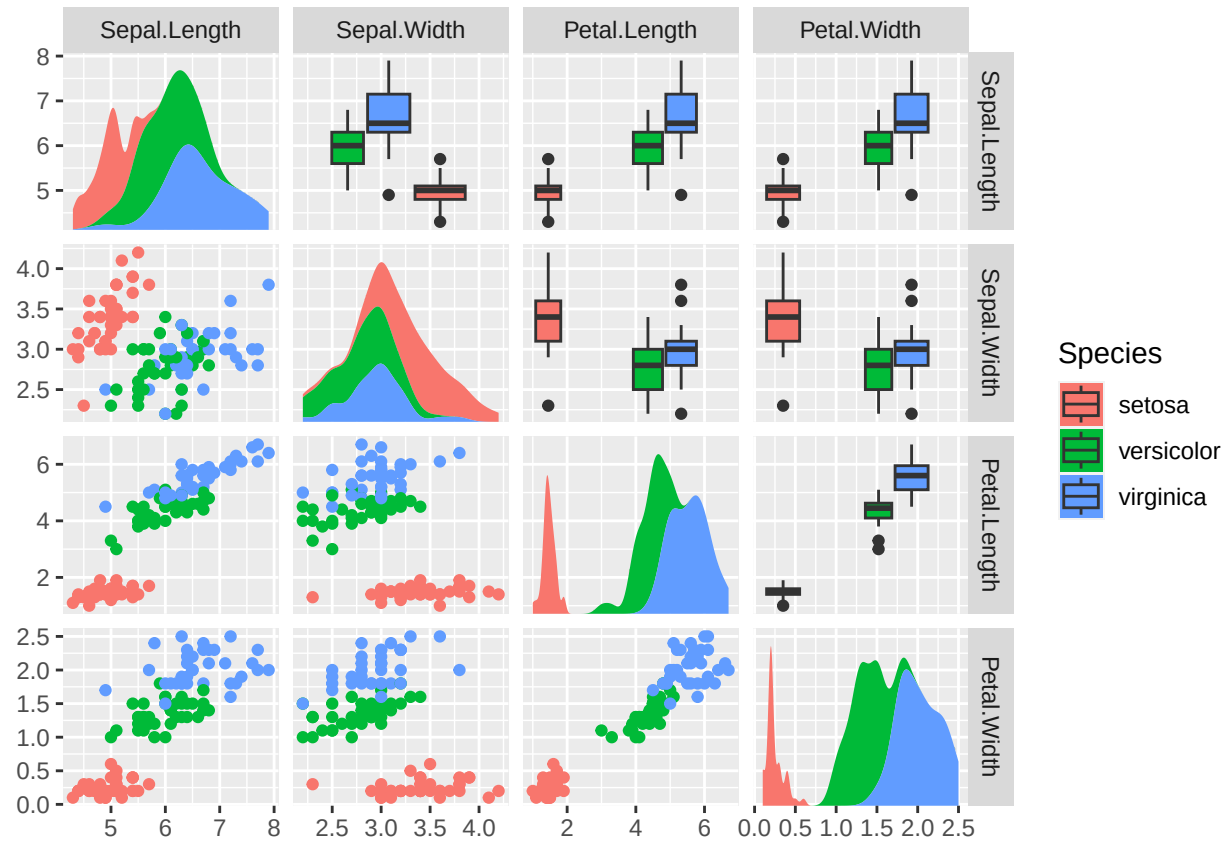
```
test
```

```
## # A tibble: 38 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##   <dbl>         <dbl>         <dbl>         <dbl> <fct>
## 1         5.8         4         1.2         0.2 setosa
## 2         5.7         4.4         1.5         0.4 setosa
## 3         5.1         3.7         1.5         0.4 setosa
## 4         5.2         3.5         1.5         0.2 setosa
## 5         4.8         3.1         1.6         0.2 setosa
## 6         5.5         3.5         1.3         0.2 setosa
## 7         5         3.5         1.3         0.3 setosa
## 8         4.6         3.2         1.4         0.2 setosa
## 9         5.3         3.7         1.5         0.2 setosa
## 10        7         3.2         4.7         1.4 versicolor
## # i 28 more rows
```

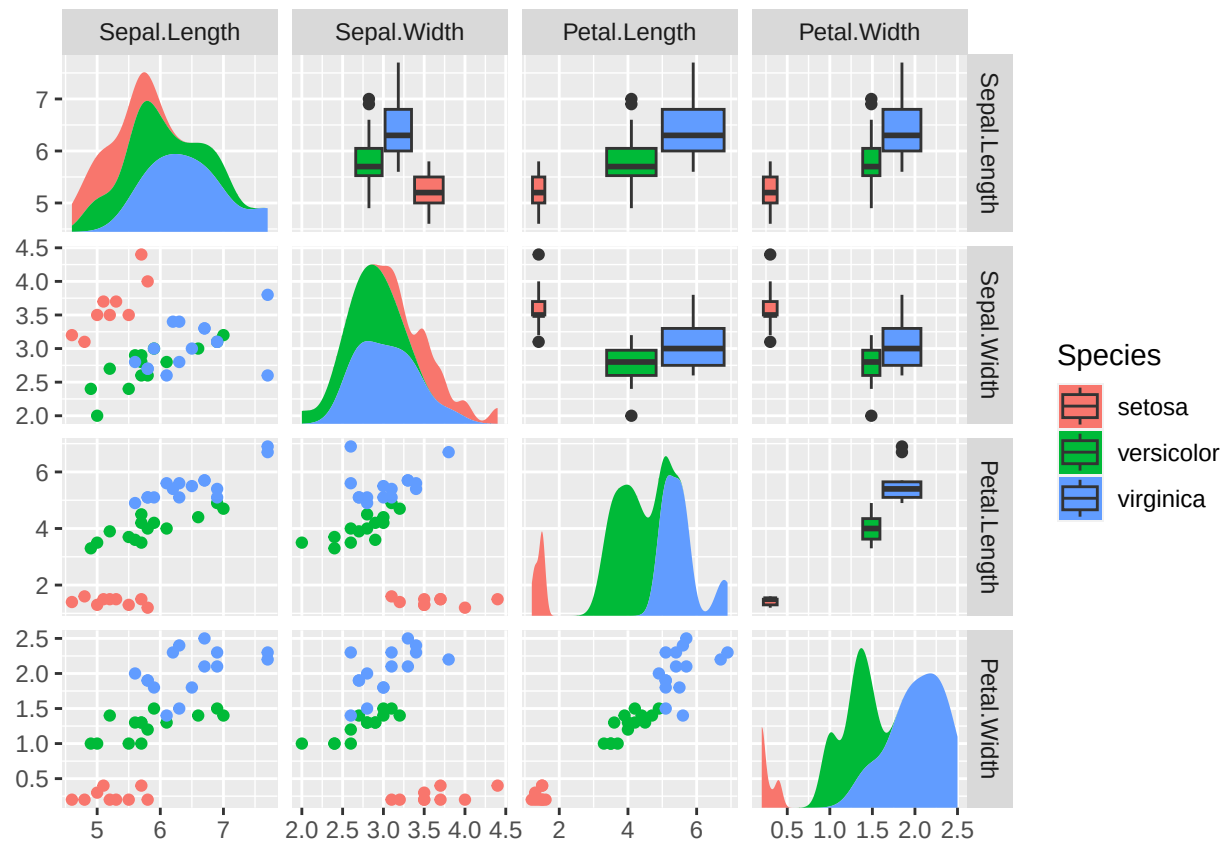
```
ggplot(data = irises, aes(x = .panel_x, y = .panel_y)) +
  geom_point(aes(color = Species)) +
  geom_autodensity(aes(fill = Species)) +
  geom_boxplot(aes(fill = Species)) +
  facet_matrix(vars(Sepal.Length:Petal.Width),
    layer.lower = 1,
    layer.diag = 2,
    layer.upper = 3)
```



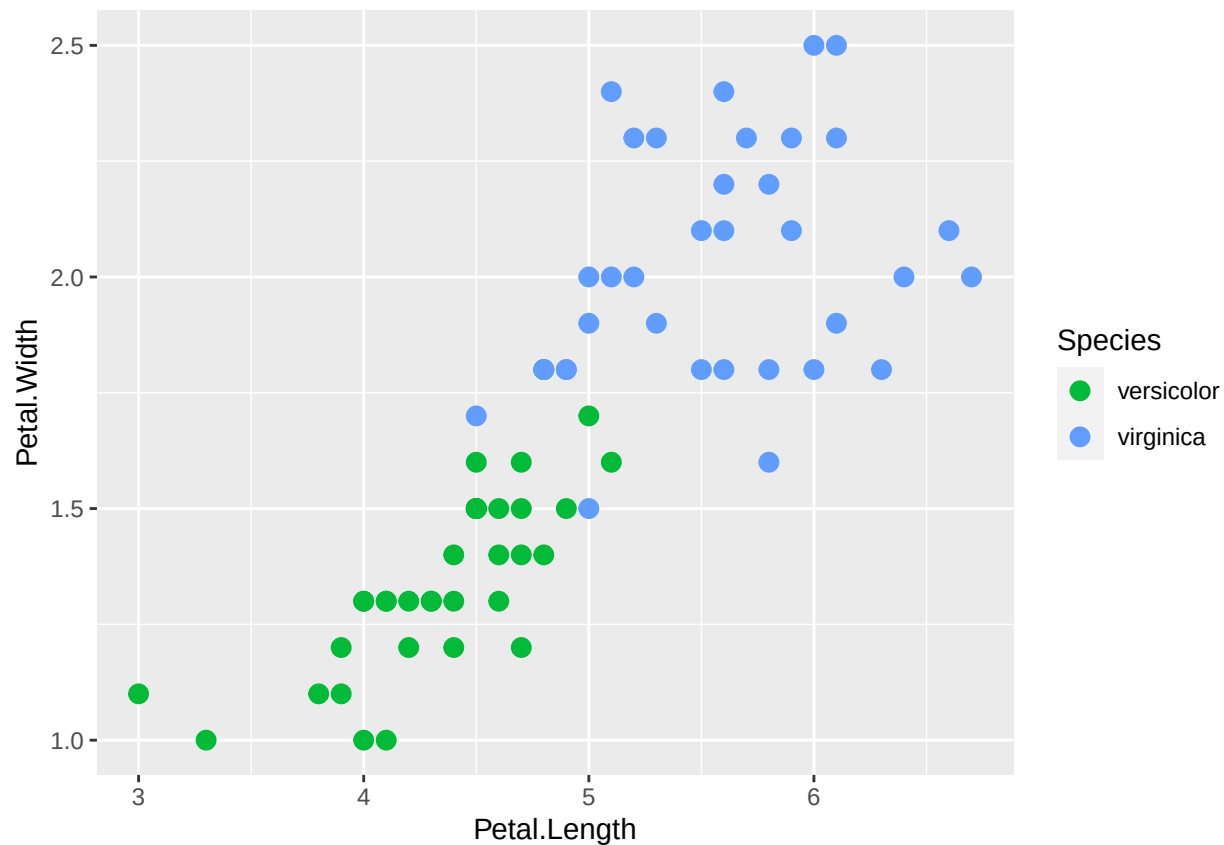
```
ggplot(data = train, aes(x = .panel_x, y = .panel_y)) +
  geom_point(aes(color = Species)) +
  geom_autodensity(aes(fill = Species)) +
  geom_boxplot(aes(fill = Species)) +
  facet_matrix(vars(Sepal.Length:Petal.Width),
    layer.lower = 1,
    layer.diag = 2,
    layer.upper = 3)
```



```
ggplot(data = test, aes(x = .panel_x, y = .panel_y)) +
  geom_point(aes(color = Species)) +
  geom_autodensity(aes(fill = Species)) +
  geom_boxplot(aes(fill = Species)) +
  facet_matrix(vars(Sepal.Length:Petal.Width),
    layer.lower = 1,
    layer.diag = 2,
    layer.upper = 3)
```

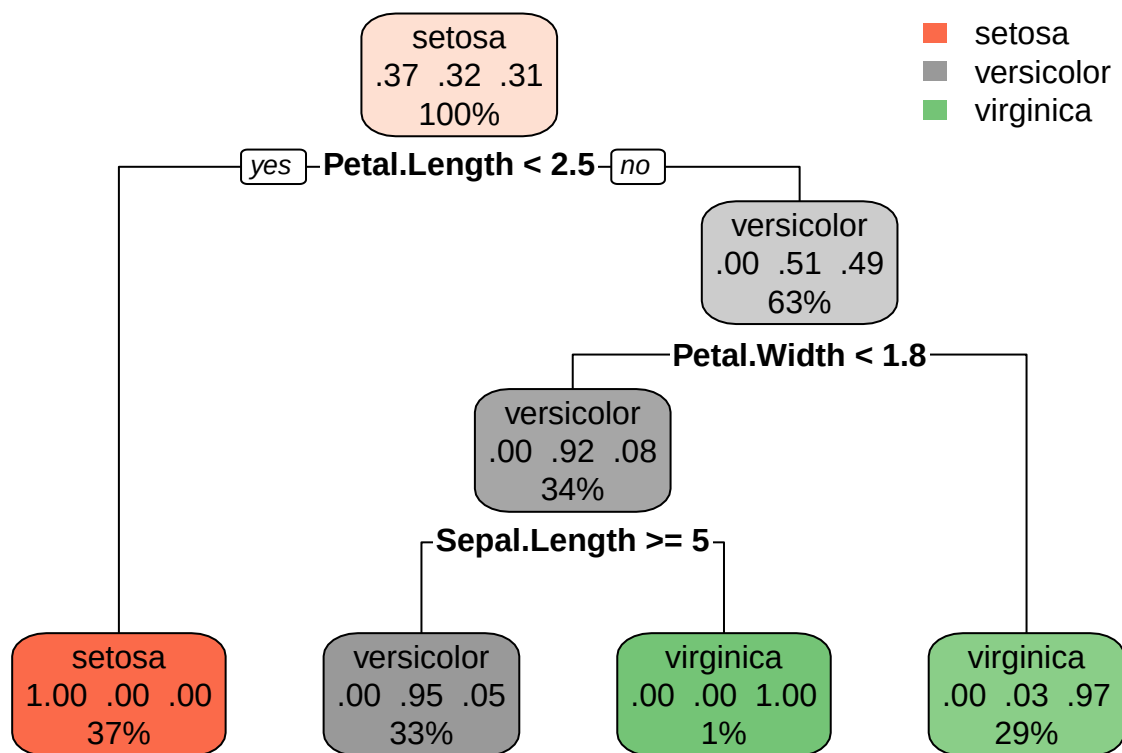


```
ggplot(filter(train, Species != "setosa")) + geom_point(aes(x = Petal.Length, y = Petal.Width, color = Species))
```



```
dtree_mod <- decision_tree(mode = "classification", min_n = 4, tree_depth = 3)
dtree_mod_fit <- fit(object = dtree_mod, formula = Species ~ ., data = train)
dtree_mod_fit %>% extract_fit_engine() %>% rpart.plot()
```

```
## Warning: Cannot retrieve the data used to build the model (so cannot determine roundint and is.binary)
## To silence this warning:
##   Call rpart.plot with roundint=FALSE,
##   or rebuild the rpart model with model=TRUE.
```



```
predictions <- predict(dtrees_mod_fit, new_data = test)
```

```
confusionMatrix(data = predictions$.pred_class, reference = test$Species)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction  setosa versicolor virginica
```

```
##   setosa      9         0         0
```

```
##   versicolor  0        13         2
```

```
##   virginica   0         1        13
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9211
```

```
##           95% CI : (0.7862, 0.9834)
```

```
##   No Information Rate : 0.3947
```

```
##   P-Value [Acc > NIR] : 1.467e-11
```

```
##
```

```
##           Kappa : 0.8791
```

```
##
```

```
##   McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
```

	Class: setosa	Class: versicolor	Class: virginica
## Sensitivity	1.0000	0.9286	0.8667
## Specificity	1.0000	0.9167	0.9565
## Pos Pred Value	1.0000	0.8667	0.9286
## Neg Pred Value	1.0000	0.9565	0.9167
## Prevalence	0.2368	0.3684	0.3947
## Detection Rate	0.2368	0.3421	0.3421
## Detection Prevalence	0.2368	0.3947	0.3684
## Balanced Accuracy	1.0000	0.9226	0.9116

```
dtree_mod_fit %>% extract_fit_engine() %>% varImp()
```

```
## Overall
## Petal.Length 65.56923
## Petal.Width 67.93858
## Sepal.Length 41.18411
## Sepal.Width 17.07716
```

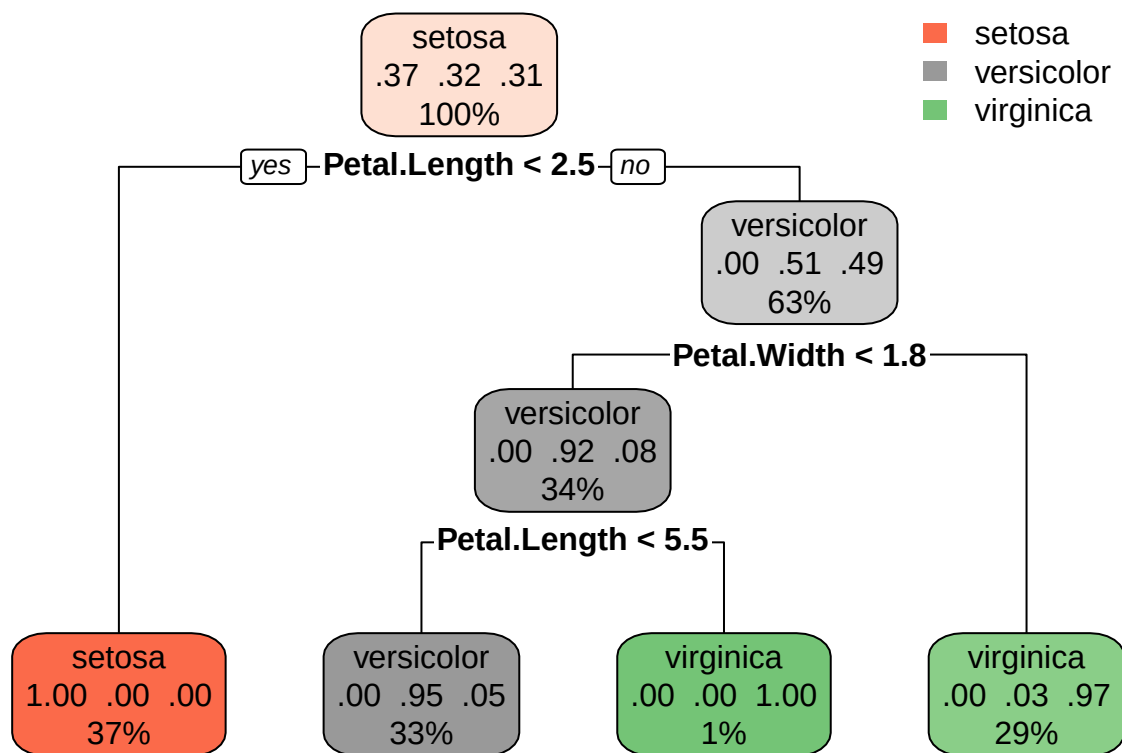
```
dtree_mod_fit2 <- dtree_mod %>% fit(Species ~ Petal.Length + Petal.Width, data = train)
```

```
dtree_mod_fit2
```

```
## parsnip model object
##
## n= 112
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 112 71 setosa (0.36607143 0.32142857 0.31250000)
##    2) Petal.Length< 2.45 41 0 setosa (1.00000000 0.00000000 0.00000000) *
##    3) Petal.Length>=2.45 71 35 versicolor (0.00000000 0.50704225 0.49295775)
##      6) Petal.Width< 1.75 38 3 versicolor (0.00000000 0.92105263 0.07894737)
##        12) Petal.Length< 5.45 37 2 versicolor (0.00000000 0.94594595 0.05405405) *
##        13) Petal.Length>=5.45 1 0 virginica (0.00000000 0.00000000 1.00000000) *
##        7) Petal.Width>=1.75 33 1 virginica (0.00000000 0.03030303 0.96969697) *
```

```
dtree_mod_fit2 %>% extract_fit_engine() %>% rpart.plot()
```

```
## Warning: Cannot retrieve the data used to build the model (so cannot determine roundint and is.binary)
## To silence this warning:
##   Call rpart.plot with roundint=FALSE,
##   or rebuild the rpart model with model=TRUE.
```



```
predictions2 <- predict(dtree_mod_fit2, new_data = test)
```

```
confusionMatrix(data = predictions2$.pred_class, reference = test$Species)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction  setosa versicolor virginica
```

```
##   setosa      9         0         0
```

```
##  versicolor  0         14         1
```

```
##  virginica   0         0        14
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9737
```

```
##           95% CI : (0.8619, 0.9993)
```

```
##   No Information Rate : 0.3947
```

```
##   P-Value [Acc > NIR] : 2.707e-14
```

```
##
```

```
##           Kappa : 0.9597
```

```
##
```

```
##   McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
```

##	Class: setosa	Class: versicolor	Class: virginica
## Sensitivity	1.0000	1.0000	0.9333
## Specificity	1.0000	0.9583	1.0000
## Pos Pred Value	1.0000	0.9333	1.0000
## Neg Pred Value	1.0000	1.0000	0.9583
## Prevalence	0.2368	0.3684	0.3947
## Detection Rate	0.2368	0.3684	0.3684
## Detection Prevalence	0.2368	0.3947	0.3684
## Balanced Accuracy	1.0000	0.9792	0.9667

```
dtree_mod_fit2 %>% extract_fit_engine() %>% varImp()
```

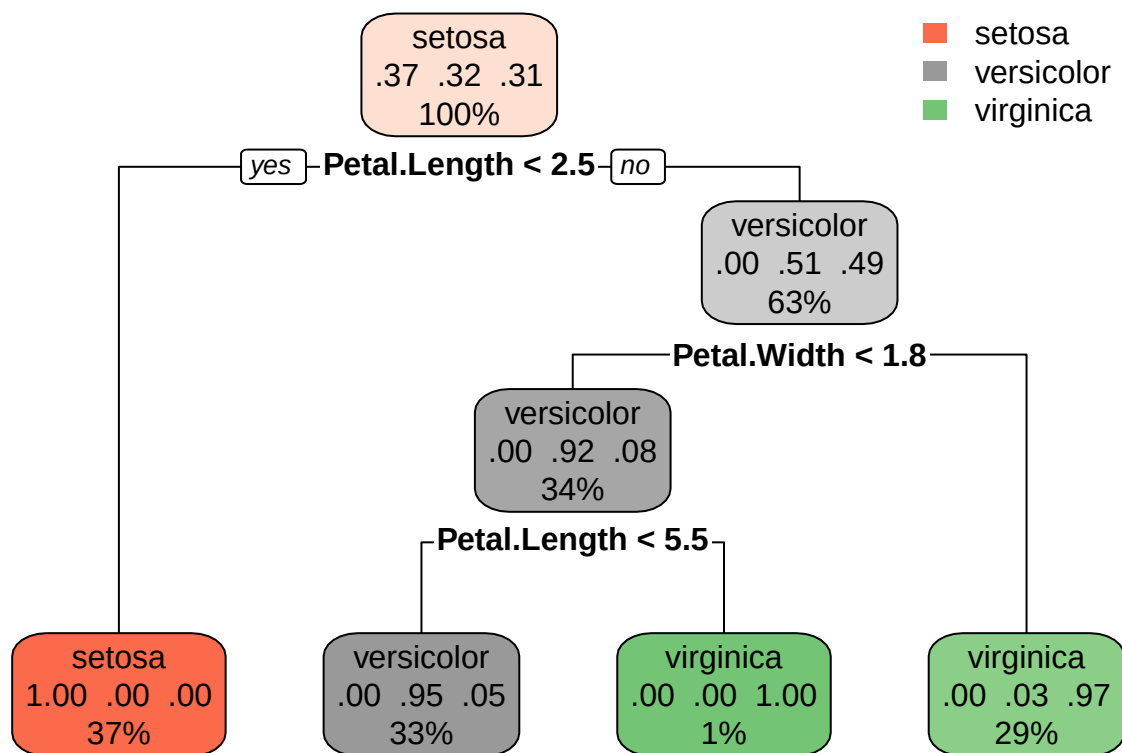
```
## Overall
## Petal.Length 65.56923
## Petal.Width 67.93858
```

```
dtree_mod_hyper <- decision_tree(mode = "classification",
                                  min_n = 1, tree_depth = 4,
                                  cost_complexity = 0)

dtree_mod_hyper_fit <- dtree_mod_hyper %>%
  fit(Species ~ Petal.Length + Petal.Width, data = train)

dtree_mod_hyper_fit %>% extract_fit_engine() %>% rpart.plot()
```

```
## Warning: Cannot retrieve the data used to build the model (so cannot determine roundint and is.binary)
## To silence this warning:
##   Call rpart.plot with roundint=FALSE,
##   or rebuild the rpart model with model=TRUE.
```



```
#Cohen's Kappa
#<= 0.2, no reliability
#0.21 to 0.39, minimal reliability
#0.40 to 0.59, weak reliability
#0.60 to 0.79, moderate reliability
#0.80 to 0.90, strong reliability
#>0.90, nearly perfect reliability

predictions <- predict(dtree_mod_fit, new_data = test)

confusionMatrix(data = predictions$.pred_class, reference = test$Species)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  setosa versicolor virginica
## setosa      9         0         0
## versicolor  0        13         2
## virginica   0         1        13
##
## Overall Statistics
##
##           Accuracy : 0.9211
##           95% CI : (0.7862, 0.9834)
##           No Information Rate : 0.3947
```

```
##      P-Value [Acc > NIR] : 1.467e-11
##
##              Kappa : 0.8791
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: setosa Class: versicolor Class: virginica
## Sensitivity          1.0000          0.9286          0.8667
## Specificity          1.0000          0.9167          0.9565
## Pos Pred Value       1.0000          0.8667          0.9286
## Neg Pred Value       1.0000          0.9565          0.9167
## Prevalence           0.2368          0.3684          0.3947
## Detection Rate       0.2368          0.3421          0.3421
## Detection Prevalence 0.2368          0.3947          0.3684
## Balanced Accuracy    1.0000          0.9226          0.9116
```

```
predictions_hyper <- predict(dtree_mod_hyper_fit, new_data = test)

confusionMatrix(data = predictions_hyper$.pred_class, reference = test$Species)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  setosa versicolor virginica
##   setosa      9         0         0
##   versicolor  0        14         1
##   virginica   0         0        14
##
## Overall Statistics
##
##              Accuracy : 0.9737
##              95% CI : (0.8619, 0.9993)
##      No Information Rate : 0.3947
##      P-Value [Acc > NIR] : 2.707e-14
##
##              Kappa : 0.9597
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: setosa Class: versicolor Class: virginica
## Sensitivity          1.0000          1.0000          0.9333
## Specificity          1.0000          0.9583          1.0000
## Pos Pred Value       1.0000          0.9333          1.0000
## Neg Pred Value       1.0000          1.0000          0.9583
## Prevalence           0.2368          0.3684          0.3947
## Detection Rate       0.2368          0.3684          0.3684
## Detection Prevalence 0.2368          0.3947          0.3684
## Balanced Accuracy    1.0000          0.9792          0.9667
```

```
folds <- vfold_cv(train, v =5 )
```

```
folds
```

```
## # 5-fold cross-validation
## # A tibble: 5 x 2
##   splits      id
##   <list>      <chr>
## 1 <split [89/23]> Fold1
## 2 <split [89/23]> Fold2
## 3 <split [90/22]> Fold3
## 4 <split [90/22]> Fold4
## 5 <split [90/22]> Fold5
```

```
dtree_mod <- decision_tree(mode = "classification")
```

```
tuned <- tune_grid(dtree_mod, preprocessor = Species ~., resamples = folds)
```

```
## Warning: No tuning parameters have been detected, performance will be evaluated
## using the resamples with no tuning. Did you want to [tune()] parameters?
```

```
tuned
```

```
## # Tuning results
## # 5-fold cross-validation
## # A tibble: 5 x 4
##   splits      id   .metrics      .notes
##   <list>      <chr> <list>      <list>
## 1 <split [89/23]> Fold1 <tibble [2 x 4]> <tibble [0 x 3]>
## 2 <split [89/23]> Fold2 <tibble [2 x 4]> <tibble [0 x 3]>
## 3 <split [90/22]> Fold3 <tibble [2 x 4]> <tibble [0 x 3]>
## 4 <split [90/22]> Fold4 <tibble [2 x 4]> <tibble [0 x 3]>
## 5 <split [90/22]> Fold5 <tibble [2 x 4]> <tibble [0 x 3]>
```

```
tuned$.metrics
```

```
## [[1]]
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>      <dbl> <chr>
## 1 accuracy multiclass    0.957 Preprocessor1_Model1
## 2 roc_auc  hand_till      0.972 Preprocessor1_Model1
##
## [[2]]
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>      <dbl> <chr>
## 1 accuracy multiclass    0.913 Preprocessor1_Model1
## 2 roc_auc  hand_till      0.942 Preprocessor1_Model1
##
## [[3]]
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>       <dbl> <chr>
## 1 accuracy multiclass    0.955 Preprocessor1_Model1
## 2 roc_auc  hand_till      0.983 Preprocessor1_Model1
##
## [[4]]
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>       <dbl> <chr>
## 1 accuracy multiclass    0.955 Preprocessor1_Model1
## 2 roc_auc  hand_till      0.967 Preprocessor1_Model1
##
## [[5]]
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>       <dbl> <chr>
## 1 accuracy multiclass      1 Preprocessor1_Model1
## 2 roc_auc  hand_till      1 Preprocessor1_Model1
```

```
collect_metrics(tuned)
```

```
## # A tibble: 2 x 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>       <dbl> <int>   <dbl> <chr>
## 1 accuracy multiclass 0.956      5 0.0138 Preprocessor1_Model1
## 2 roc_auc  hand_till  0.973      5 0.00964 Preprocessor1_Model1
```

```
tunable(dtree_mod)
```

```
## # A tibble: 3 x 5
##   name      call_info      source      component      component_id
##   <chr>    <list>       <chr>      <chr>      <chr>
## 1 tree_depth <named list [2]> model_spec decision_tree main
## 2 min_n      <named list [2]> model_spec decision_tree main
## 3 cost_complexity <named list [2]> model_spec decision_tree main
```

```
dtree_mod_tune <- decision_tree(mode = "classification", tree_depth = tune(), min_n = tune(), cost_comp
```

```
tree_grid_auto <- grid_regular(tree_depth(), min_n(), cost_complexity())
```

```
tree_grid_auto
```

```
## # A tibble: 27 x 3
##   tree_depth min_n cost_complexity
##   <int> <int>       <dbl>
## 1      1      2  0.0000000001
## 2      8      2  0.0000000001
## 3     15      2  0.0000000001
## 4      1     21  0.0000000001
## 5      8     21  0.0000000001
## 6     15     21  0.0000000001
```

```
## 7      1  40  0.0000000001
## 8      8  40  0.0000000001
## 9     15  40  0.0000000001
## 10     1   2  0.00000316
## # i 17 more rows
```

```
print(tree_grid_auto, n = 27)
```

```
## # A tibble: 27 x 3
##   tree_depth min_n cost_complexity
##   <int> <int>         <dbl>
## 1      1     2  0.0000000001
## 2      8     2  0.0000000001
## 3     15     2  0.0000000001
## 4      1    21  0.0000000001
## 5      8    21  0.0000000001
## 6     15    21  0.0000000001
## 7      1    40  0.0000000001
## 8      8    40  0.0000000001
## 9     15    40  0.0000000001
## 10     1     2  0.00000316
## 11     8     2  0.00000316
## 12    15     2  0.00000316
## 13     1    21  0.00000316
## 14     8    21  0.00000316
## 15    15    21  0.00000316
## 16     1    40  0.00000316
## 17     8    40  0.00000316
## 18    15    40  0.00000316
## 19     1     2  0.1
## 20     8     2  0.1
## 21    15     2  0.1
## 22     1    21  0.1
## 23     8    21  0.1
## 24    15    21  0.1
## 25     1    40  0.1
## 26     8    40  0.1
## 27    15    40  0.1
```

```
tree_grid_manual <- expand_grid(tree_depth = 2:5, min_n = seq(1,51,10), cost_complexity = c(0, 0.1, 0.2
```

```
tree_grid_manual
```

```
## # A tibble: 96 x 3
##   tree_depth min_n cost_complexity
##   <int> <dbl>         <dbl>
## 1      2     1         0
## 2      3     1         0
## 3      4     1         0
## 4      5     1         0
## 5      2    11         0
## 6      3    11         0
## 7      4    11         0
```

```
## 8      5    11      0
## 9      2    21      0
## 10     3    21      0
## # i 86 more rows
```

```
print(tree_grid_manual, n = 2967)
```

```
## # A tibble: 96 x 3
##   tree_depth min_n cost_complexity
##   <int> <dbl> <dbl>
## 1      2      1      0
## 2      3      1      0
## 3      4      1      0
## 4      5      1      0
## 5      2     11      0
## 6      3     11      0
## 7      4     11      0
## 8      5     11      0
## 9      2     21      0
## 10     3     21      0
## 11     4     21      0
## 12     5     21      0
## 13     2     31      0
## 14     3     31      0
## 15     4     31      0
## 16     5     31      0
## 17     2     41      0
## 18     3     41      0
## 19     4     41      0
## 20     5     41      0
## 21     2     51      0
## 22     3     51      0
## 23     4     51      0
## 24     5     51      0
## 25     2      1     0.1
## 26     3      1     0.1
## 27     4      1     0.1
## 28     5      1     0.1
## 29     2     11     0.1
## 30     3     11     0.1
## 31     4     11     0.1
## 32     5     11     0.1
## 33     2     21     0.1
## 34     3     21     0.1
## 35     4     21     0.1
## 36     5     21     0.1
## 37     2     31     0.1
## 38     3     31     0.1
## 39     4     31     0.1
## 40     5     31     0.1
## 41     2     41     0.1
## 42     3     41     0.1
## 43     4     41     0.1
## 44     5     41     0.1
```

## 45	2	51	0.1
## 46	3	51	0.1
## 47	4	51	0.1
## 48	5	51	0.1
## 49	2	1	0.2
## 50	3	1	0.2
## 51	4	1	0.2
## 52	5	1	0.2
## 53	2	11	0.2
## 54	3	11	0.2
## 55	4	11	0.2
## 56	5	11	0.2
## 57	2	21	0.2
## 58	3	21	0.2
## 59	4	21	0.2
## 60	5	21	0.2
## 61	2	31	0.2
## 62	3	31	0.2
## 63	4	31	0.2
## 64	5	31	0.2
## 65	2	41	0.2
## 66	3	41	0.2
## 67	4	41	0.2
## 68	5	41	0.2
## 69	2	51	0.2
## 70	3	51	0.2
## 71	4	51	0.2
## 72	5	51	0.2
## 73	2	1	0.3
## 74	3	1	0.3
## 75	4	1	0.3
## 76	5	1	0.3
## 77	2	11	0.3
## 78	3	11	0.3
## 79	4	11	0.3
## 80	5	11	0.3
## 81	2	21	0.3
## 82	3	21	0.3
## 83	4	21	0.3
## 84	5	21	0.3
## 85	2	31	0.3
## 86	3	31	0.3
## 87	4	31	0.3
## 88	5	31	0.3
## 89	2	41	0.3
## 90	3	41	0.3
## 91	4	41	0.3
## 92	5	41	0.3
## 93	2	51	0.3
## 94	3	51	0.3
## 95	4	51	0.3
## 96	5	51	0.3

```
tree_grid_regular_fixed <- grid_regular(tree_depth(), min_n(), levels = 3)
```

```
tree_grid_regular_fixed
```

```
## # A tibble: 9 x 2
##   tree_depth min_n
##   <int> <int>
## 1      1      2
## 2      8      2
## 3     15      2
## 4      1     21
## 5      8     21
## 6     15     21
## 7      1     40
## 8      8     40
## 9     15     40
```

```
tuning_data <- tune_grid(dtree_mod_tune, preprocessor = Species ~., resamples = folds, grid = tree_grid_regular_fixed)
```

```
tuning_data
```

```
## # Tuning results
## # 5-fold cross-validation
## # A tibble: 5 x 4
##   splits          id .metrics          .notes
##   <list>         <chr> <list>         <list>
## 1 <split [89/23]> Fold1 <tibble [54 x 7]> <tibble [0 x 3]>
## 2 <split [89/23]> Fold2 <tibble [54 x 7]> <tibble [0 x 3]>
## 3 <split [90/22]> Fold3 <tibble [54 x 7]> <tibble [0 x 3]>
## 4 <split [90/22]> Fold4 <tibble [54 x 7]> <tibble [0 x 3]>
## 5 <split [90/22]> Fold5 <tibble [54 x 7]> <tibble [0 x 3]>
```

```
print(tuning_data %>% collect_metrics, n = 54)
```

```
## # A tibble: 54 x 9
##   cost_complexity tree_depth min_n .metric .estimator mean n std_err
##   <dbl>          <int> <int> <chr>   <chr>      <dbl> <int> <dbl>
## 1  0.0000000001      1      2 accuracy multiclass 0.572    5 0.0168
## 2  0.0000000001      1      2 roc_auc   hand_till 0.833    5 0
## 3  0.0000000001      8      2 accuracy multiclass 0.929    5 0.0173
## 4  0.0000000001      8      2 roc_auc   hand_till 0.944    5 0.0166
## 5  0.0000000001     15      2 accuracy multiclass 0.929    5 0.0173
## 6  0.0000000001     15      2 roc_auc   hand_till 0.944    5 0.0166
## 7  0.0000000001      1     21 accuracy multiclass 0.572    5 0.0168
## 8  0.0000000001      1     21 roc_auc   hand_till 0.833    5 0
## 9  0.0000000001      8     21 accuracy multiclass 0.956    5 0.0138
## 10 0.0000000001      8     21 roc_auc   hand_till 0.973    5 0.00964
## 11 0.0000000001     15     21 accuracy multiclass 0.956    5 0.0138
## 12 0.0000000001     15     21 roc_auc   hand_till 0.973    5 0.00964
## 13 0.0000000001      1     40 accuracy multiclass 0.572    5 0.0168
## 14 0.0000000001      1     40 roc_auc   hand_till 0.833    5 0
```

```

## 15 0.0000000001      8 40 accuracy multiclass 0.956 5 0.0138
## 16 0.0000000001      8 40 roc_auc hand_till 0.973 5 0.00964
## 17 0.0000000001     15 40 accuracy multiclass 0.956 5 0.0138
## 18 0.0000000001     15 40 roc_auc hand_till 0.973 5 0.00964
## 19 0.00000316        1  2 accuracy multiclass 0.572 5 0.0168
## 20 0.00000316        1  2 roc_auc hand_till 0.833 5 0
## 21 0.00000316        8  2 accuracy multiclass 0.929 5 0.0173
## 22 0.00000316        8  2 roc_auc hand_till 0.944 5 0.0166
## 23 0.00000316     15  2 accuracy multiclass 0.929 5 0.0173
## 24 0.00000316     15  2 roc_auc hand_till 0.944 5 0.0166
## 25 0.00000316        1 21 accuracy multiclass 0.572 5 0.0168
## 26 0.00000316        1 21 roc_auc hand_till 0.833 5 0
## 27 0.00000316        8 21 accuracy multiclass 0.956 5 0.0138
## 28 0.00000316        8 21 roc_auc hand_till 0.973 5 0.00964
## 29 0.00000316     15 21 accuracy multiclass 0.956 5 0.0138
## 30 0.00000316     15 21 roc_auc hand_till 0.973 5 0.00964
## 31 0.00000316        1 40 accuracy multiclass 0.572 5 0.0168
## 32 0.00000316        1 40 roc_auc hand_till 0.833 5 0
## 33 0.00000316        8 40 accuracy multiclass 0.956 5 0.0138
## 34 0.00000316        8 40 roc_auc hand_till 0.973 5 0.00964
## 35 0.00000316     15 40 accuracy multiclass 0.956 5 0.0138
## 36 0.00000316     15 40 roc_auc hand_till 0.973 5 0.00964
## 37 0.1                1  2 accuracy multiclass 0.572 5 0.0168
## 38 0.1                1  2 roc_auc hand_till 0.833 5 0
## 39 0.1                8  2 accuracy multiclass 0.956 5 0.0138
## 40 0.1                8  2 roc_auc hand_till 0.973 5 0.00964
## 41 0.1               15  2 accuracy multiclass 0.956 5 0.0138
## 42 0.1               15  2 roc_auc hand_till 0.973 5 0.00964
## 43 0.1                1 21 accuracy multiclass 0.572 5 0.0168
## 44 0.1                1 21 roc_auc hand_till 0.833 5 0
## 45 0.1                8 21 accuracy multiclass 0.956 5 0.0138
## 46 0.1                8 21 roc_auc hand_till 0.973 5 0.00964
## 47 0.1               15 21 accuracy multiclass 0.956 5 0.0138
## 48 0.1               15 21 roc_auc hand_till 0.973 5 0.00964
## 49 0.1                1 40 accuracy multiclass 0.572 5 0.0168
## 50 0.1                1 40 roc_auc hand_till 0.833 5 0
## 51 0.1                8 40 accuracy multiclass 0.956 5 0.0138
## 52 0.1                8 40 roc_auc hand_till 0.973 5 0.00964
## 53 0.1               15 40 accuracy multiclass 0.956 5 0.0138
## 54 0.1               15 40 roc_auc hand_till 0.973 5 0.00964
## # i 1 more variable: .config <chr>

```